



第六章

死锁 (Deadlock)





Review



- 进程间通信IPC
 - 两类：低级通信、高级通信
 - 低级通信：信号量机制
 - 高级通信
 - 共享存储器系统：共享数据结构、共享存储区
 - 消息系统：直接通信(OS缓存)、间接通信(邮箱)
 - 利用共享文件：管道
 - 有名管道（相关进程间）、无名管道（C/S）
 - 信号
 - 套接字socket



6 死锁问题 (DEADLOCK)

多进程（线程）**并发**，**共享**系统资源，但因为资源**有限**，会出现申请而得不到满足的情况，出现持有等待的问题，这会出现死锁情况

1. 概述
2. 进程的描述
3. 进程控制
4. 线程
5. 进程互斥和同步
6. 进程间通信
- 7. 死锁问题**
- 8. 本章小结**

问题与需求：

- 多道程序系统，多道程序并发执行，共享系统资源（软硬件资源、数据、数据结构）；
- **资源有限**，申请不到就得等待，并且持有等待，例如就像P操作顺序
- 结果：产生死锁



6 死锁问题 (DEADLOCK)



6 死锁问题 (DEADLOCK)

1. 概述
2. 死锁的预防
3. 死锁的检测
4. 死锁的避免
5. 解决死锁问题的综合方法

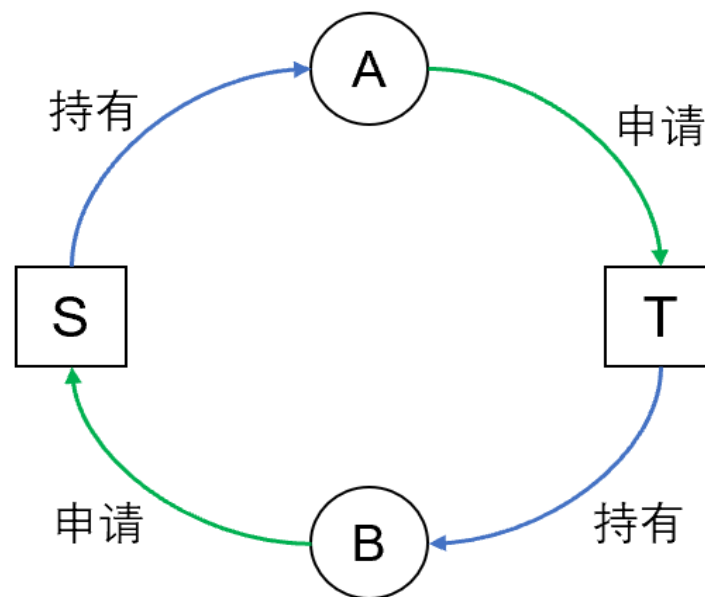
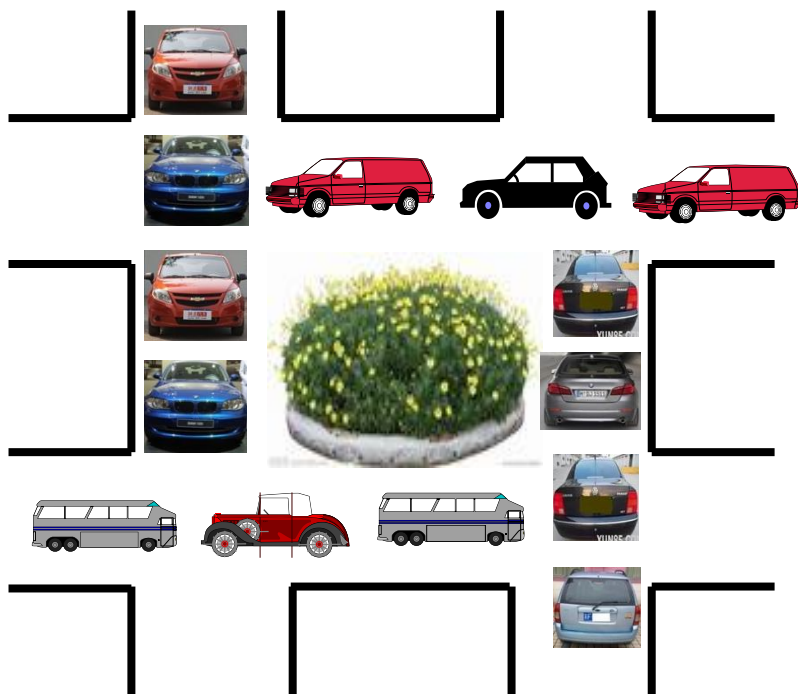
目标 (Objectives) :

- 能够叙述死锁的定义、原因、产生的条件
- 能够利用死锁预防的方法解决死锁问题
- 能够利用银行家算法, 判断系统状态是否安全
- 能够利用死锁定理, 判断系统是否处于不安全状态
- 能够比较各种死锁处理机制的优缺点
- 了解典型系统的死锁解决方案



6.1 死锁概述

(1) 死锁的现象





6.1 死锁概述

(2) 死锁的例子

进程P

...

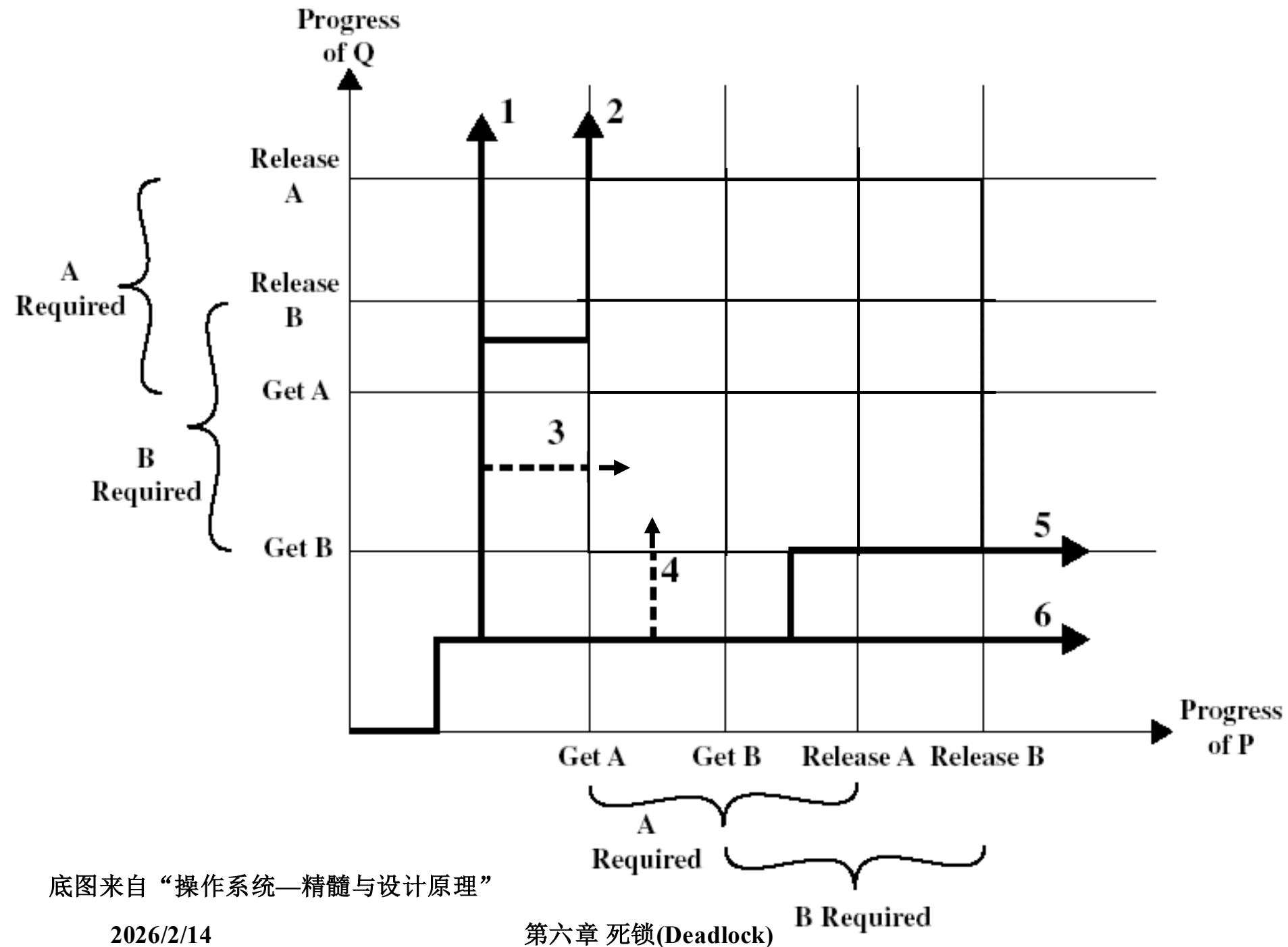
```
Request(A);    <a>
Request(B);    <b>
Release(A);
Release(B);
```

进程Q

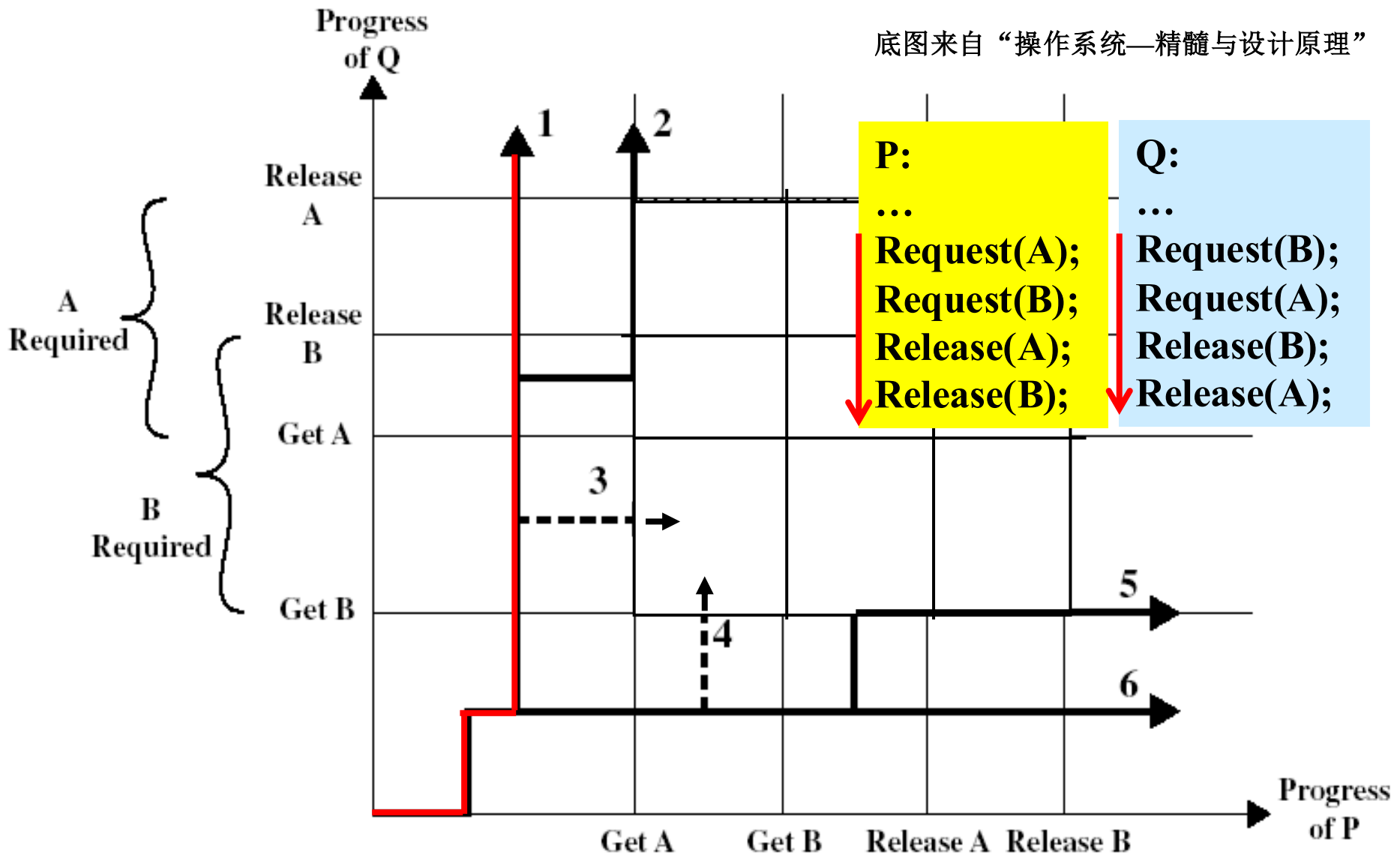
...

```
Request(B);    <b>
Request(A);    <a>
Release(B);
Release(A);
```

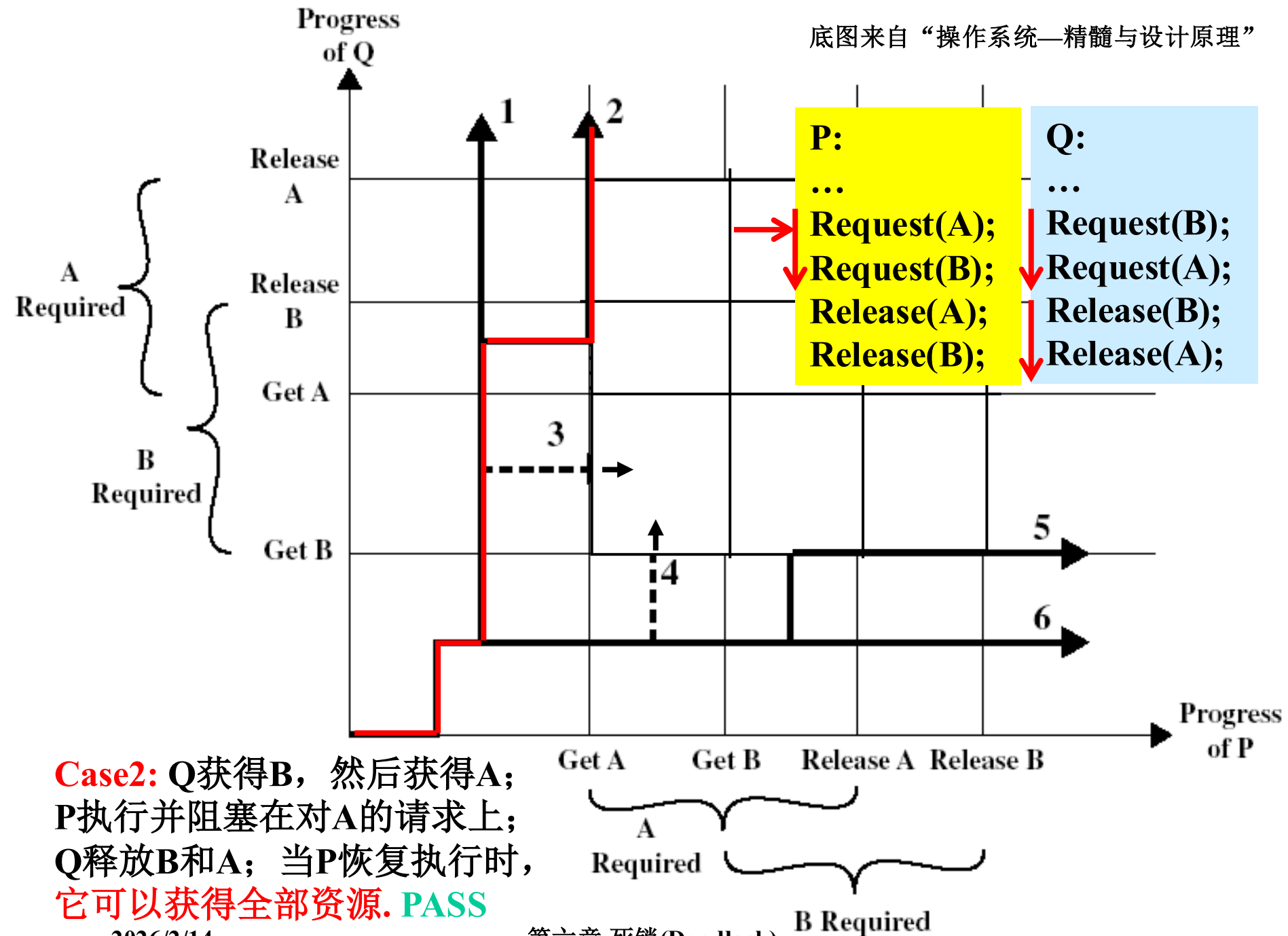
P、Q有 6 种不同的并发执行顺序：

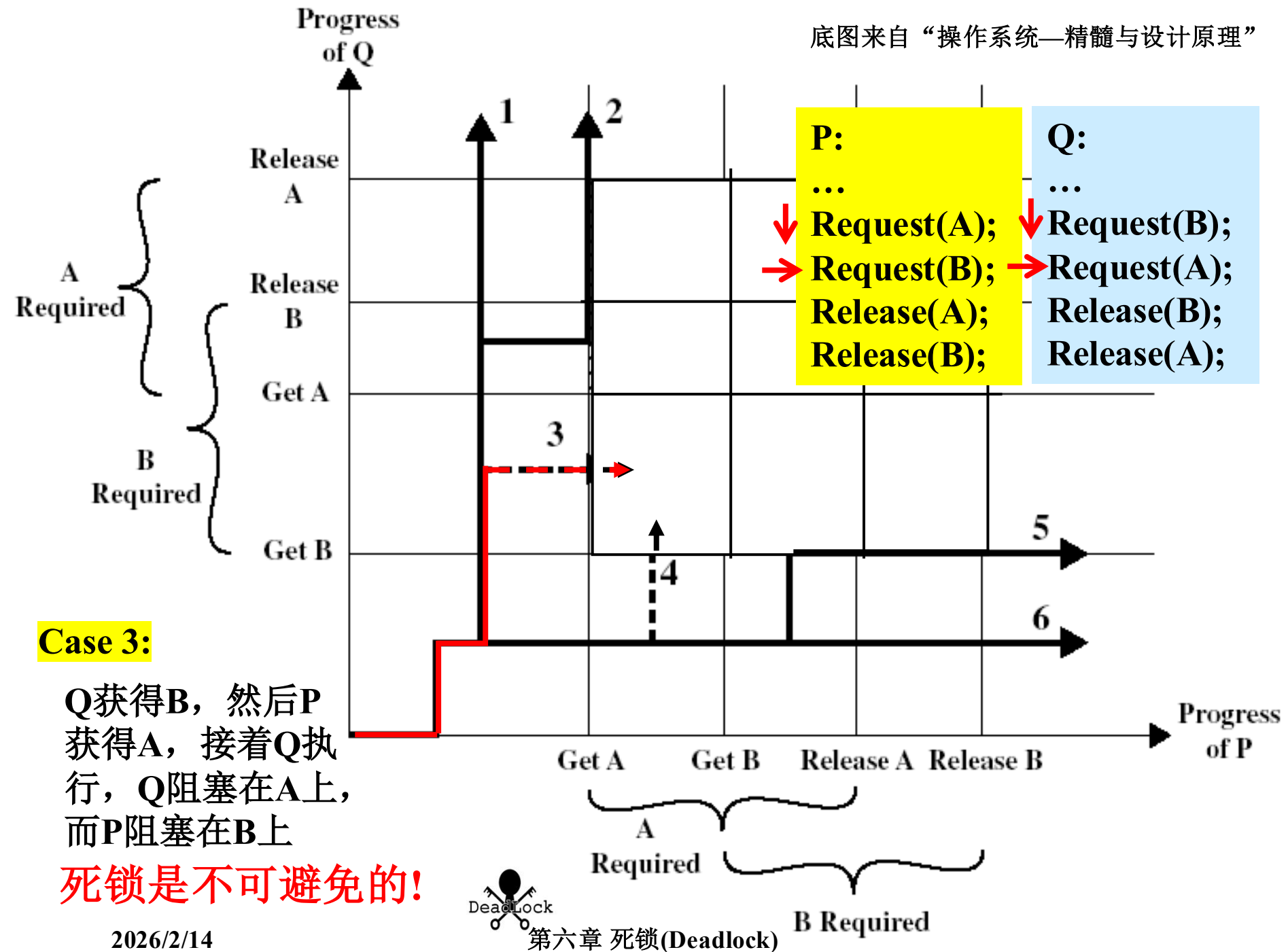


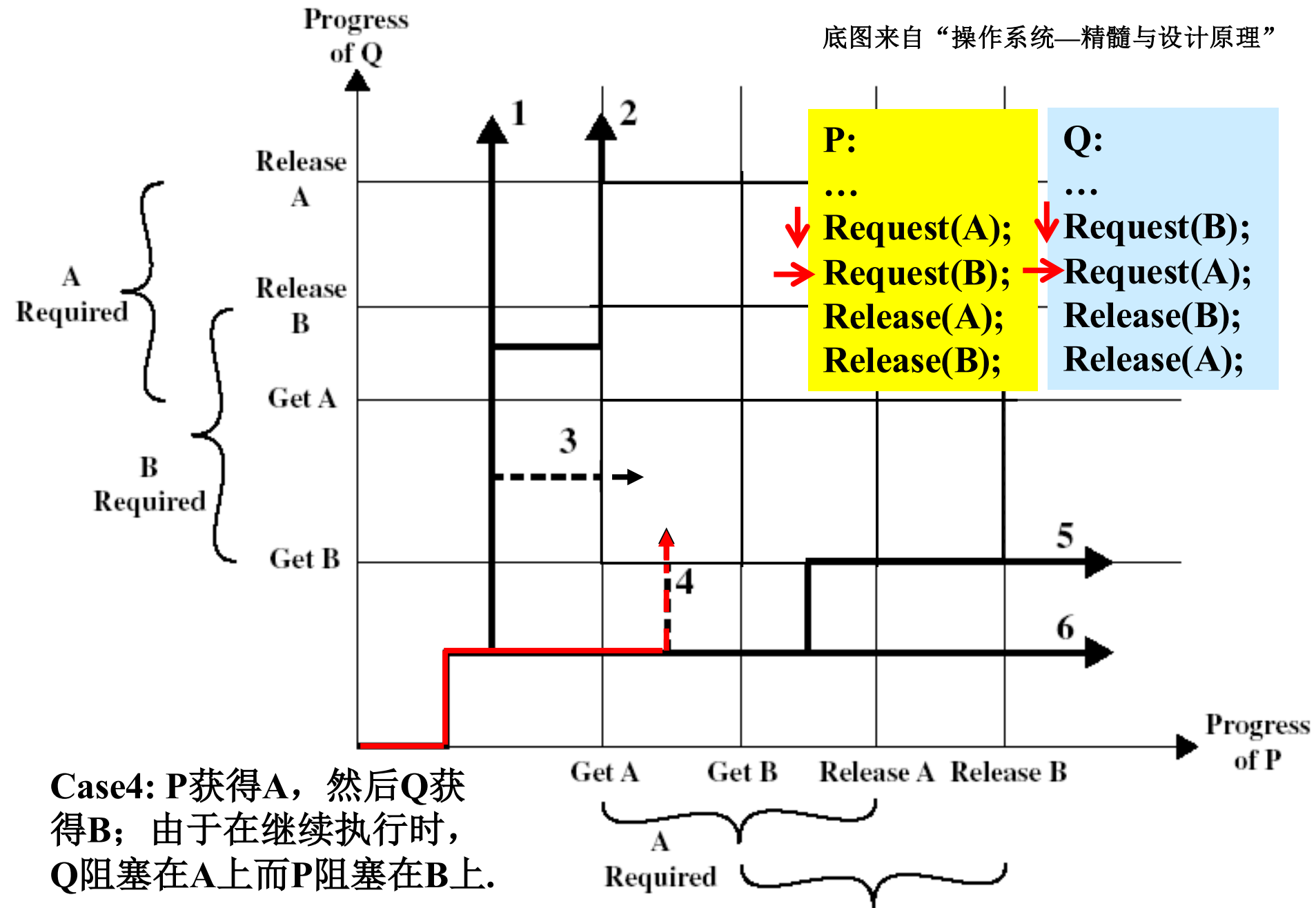
底图来自“操作系统—精髓与设计原理”



Case1: Q获得B, 然后获得A;
然后释放B和A; 当P恢复执行时,
它可以获得全部资源, PASS

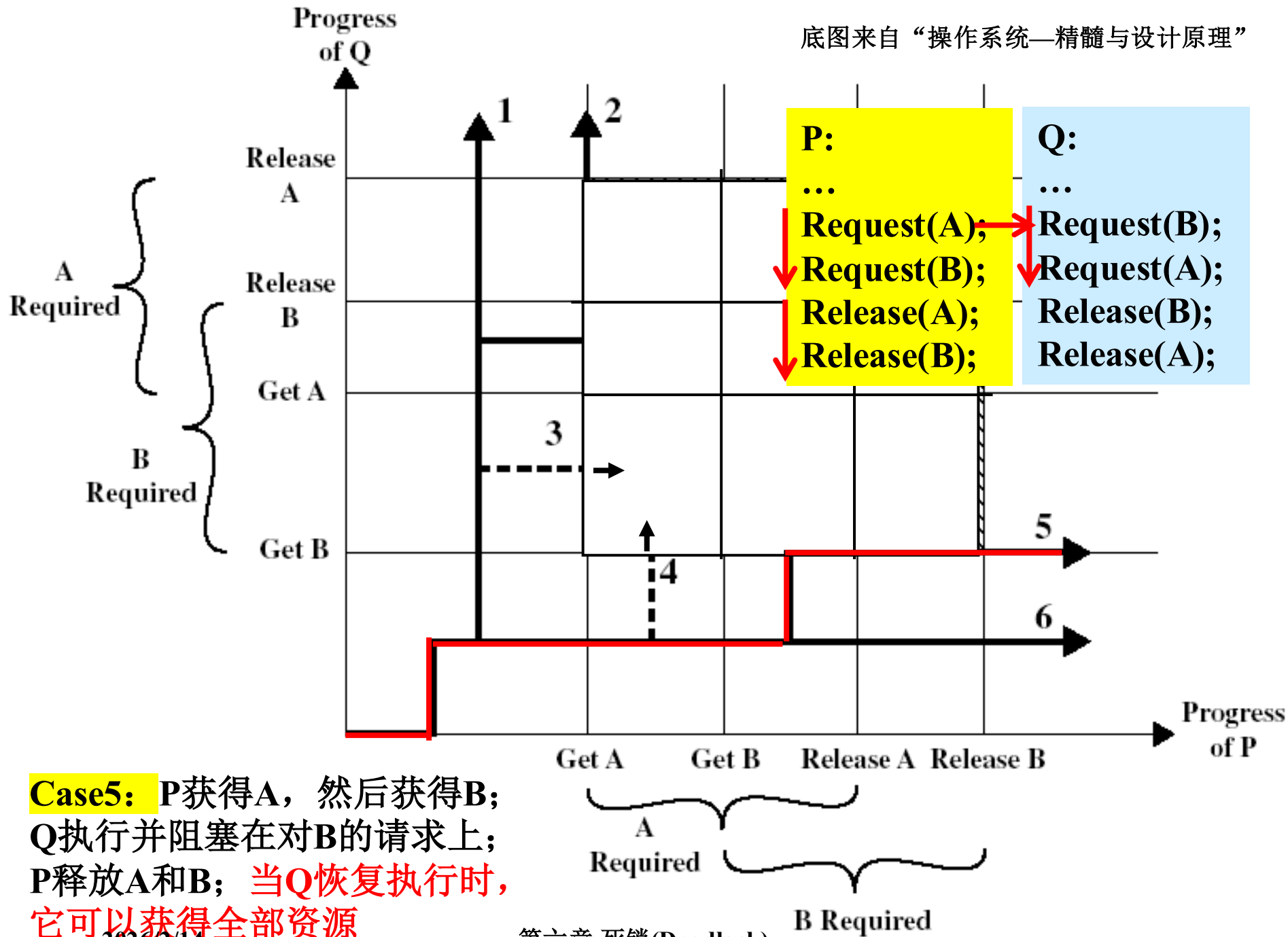


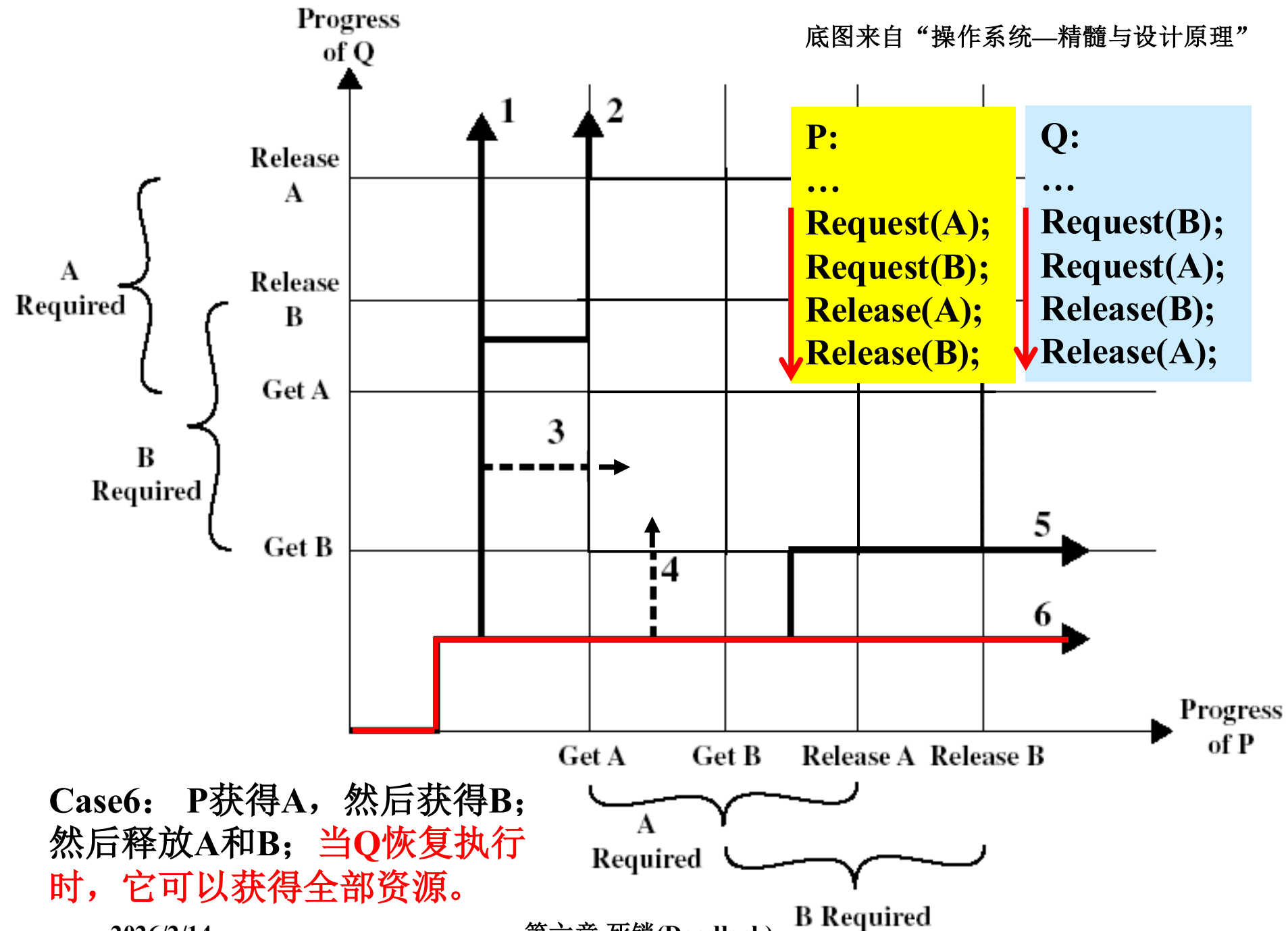


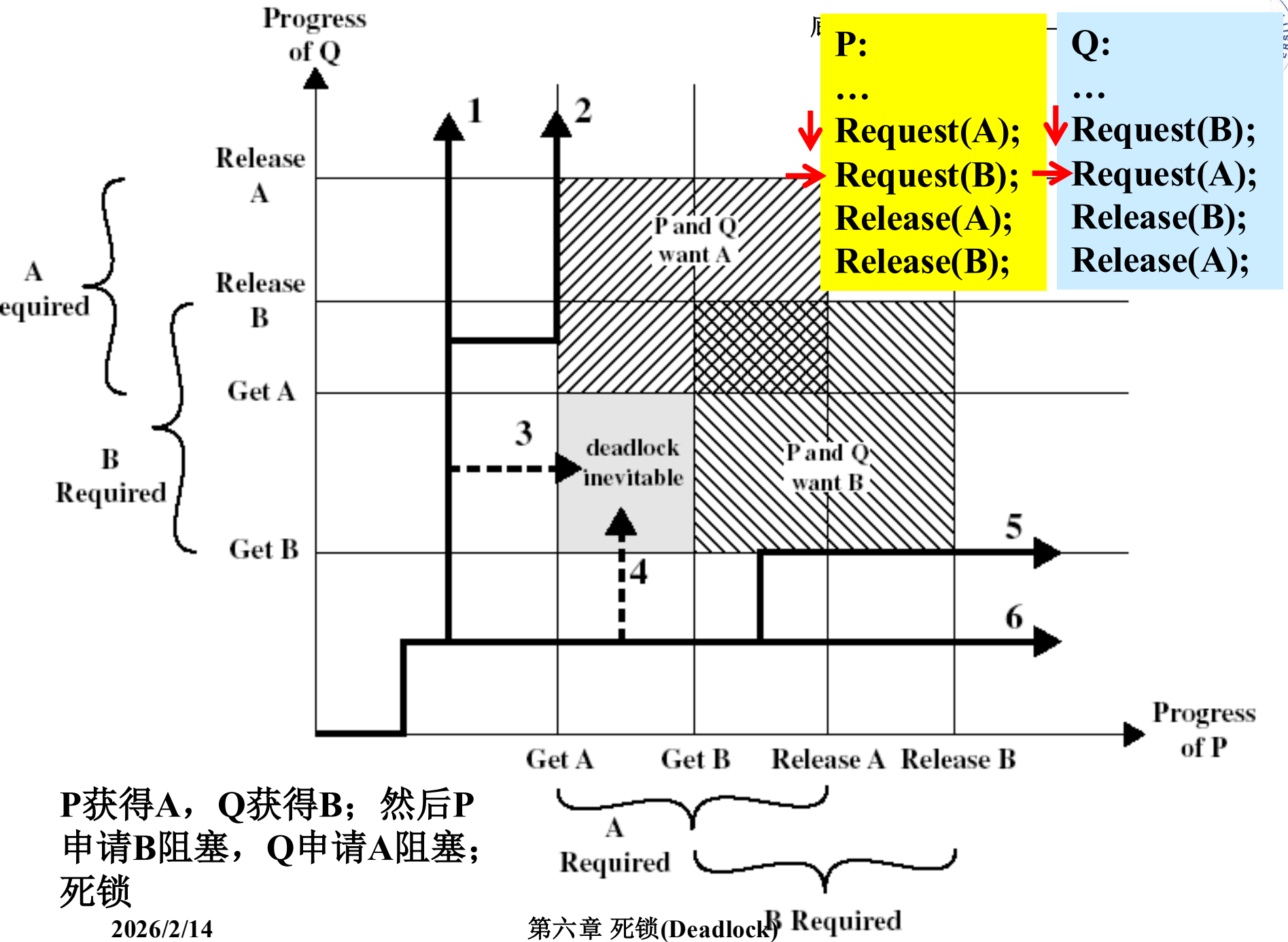


Case4: P获得A, 然后Q获得B; 由于在继续执行时, Q阻塞在A上而P阻塞在B上.

死锁是不可避免的









6.1 死锁概述

(3) 不会死锁的例子

Process P

....

Get A

.....

Release A

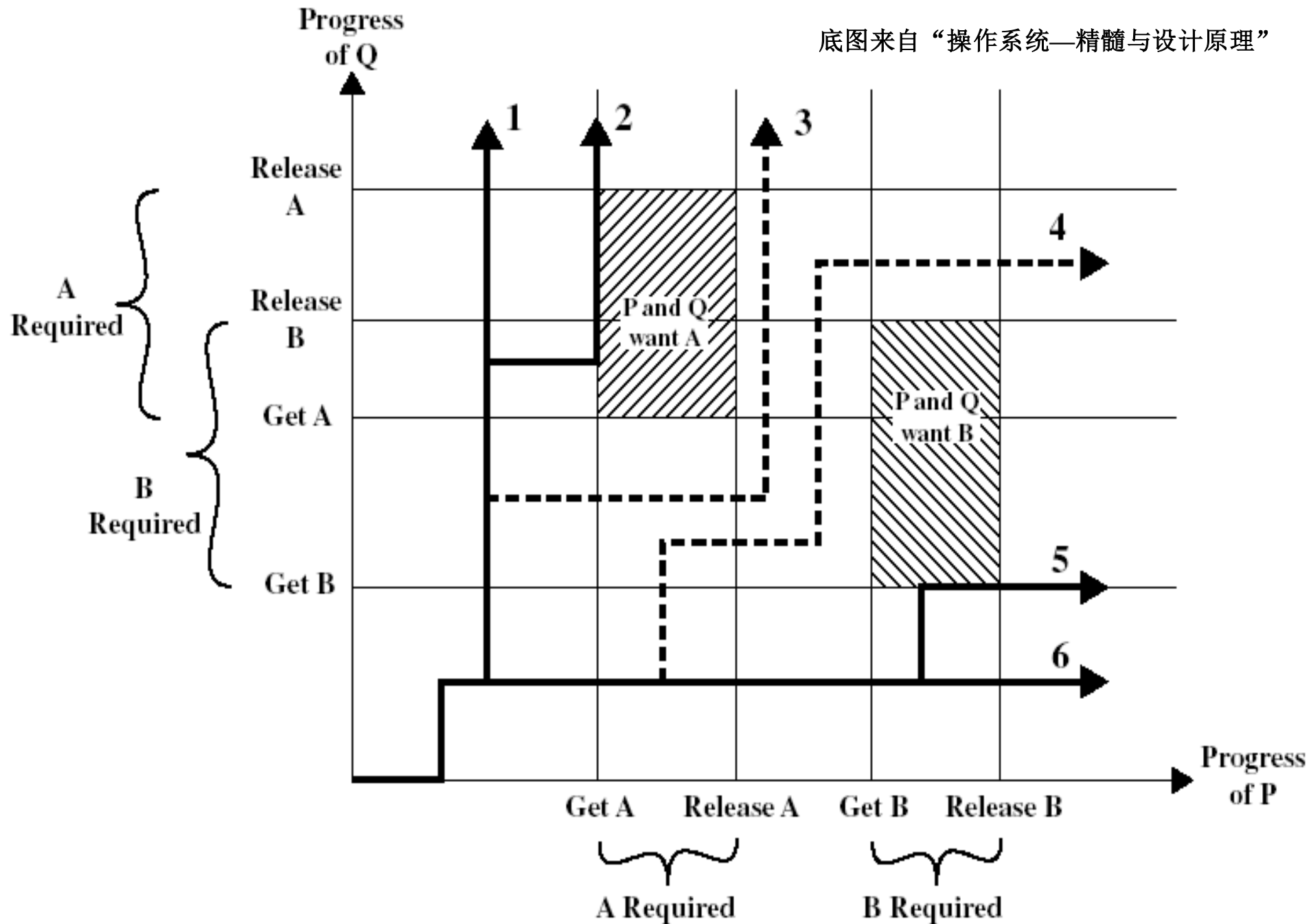
.....

Get B

.....

Release B

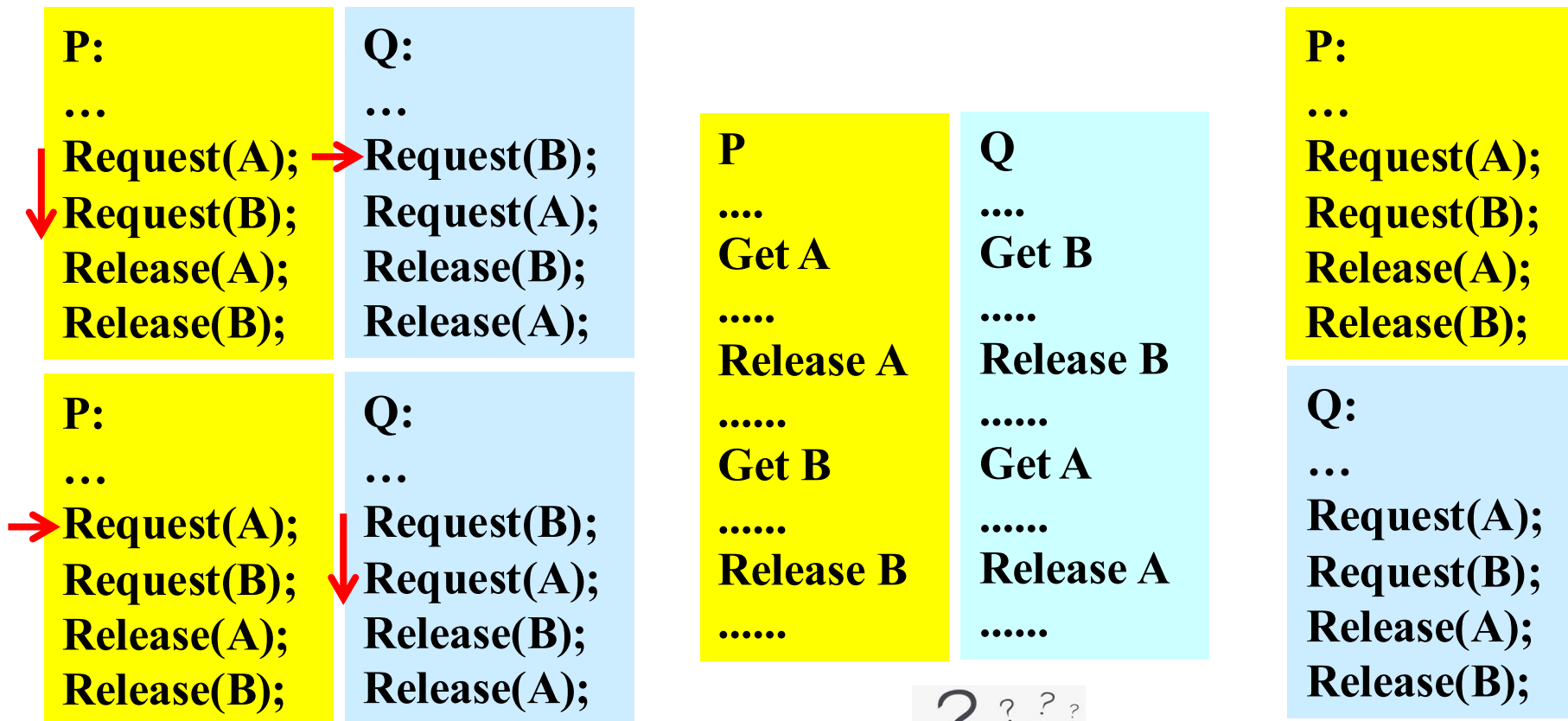
.....





6.1 死锁概述

(3) 不会死锁的情形



思考: What? 如何解释上述几种情况?

总结: 代表了死锁处理的不同策略。

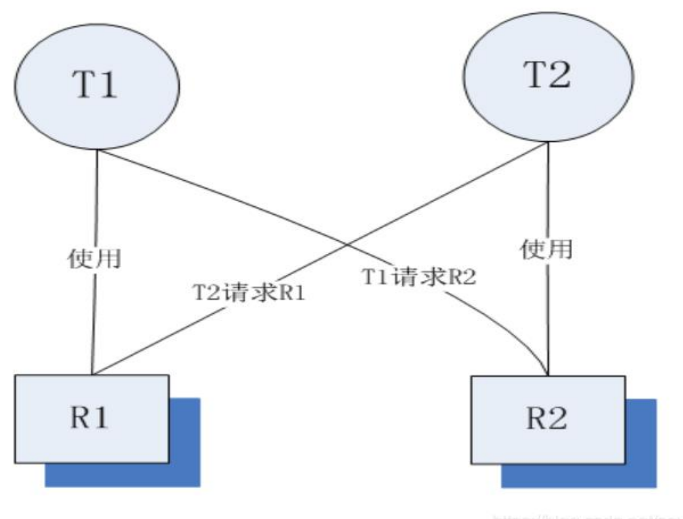


6.1 死锁概述

(4) 死锁的定义

● **定义1:** 一组并发进程中，每个进程都**无限等待**被该组进程中另一进程所占有的、而永远无法得到的资源，这种**现象**称为进程死锁，这一组进程就称为死锁进程。

● **定义2:** **死锁**是指两个或两个以上的并发进程在执行过程中，由于**竞争资源**或者由于**彼此通信**而造成的一种**阻塞的现象**，若无外力作用，它们都将无法推进下去。此时称系统处于**死锁状态**或系统产生了死锁，这些永远在互相等待的进程称为**死锁进程**【百度百科】

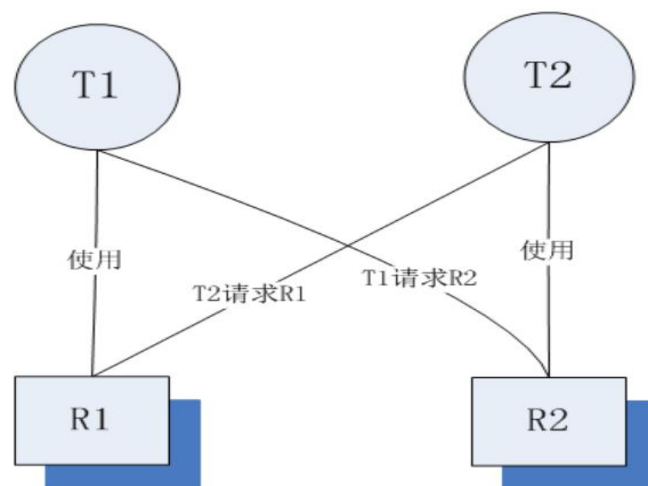




6.1 死锁概述

(5) 死锁发生的原因

- ① **竞争资源**：为多个进程所共享的资源不足，引起它们对资源的竞争而产生死锁；
- ② **并发执行的顺序不当**：进程运行过程中，请求和释放资源的顺序不当，而导致进程死锁。





6.1 死锁概述

(5) 死锁发生的原因

● 两种资源

- 可重用资源 (reusable resource)

- 可消费资源 (consumable resource)

● 资源使用模式:

“申请--分配--使用--释放” 模式



6.1 死锁概述

(5) 死锁发生的原因

● 可重用资源(reusable resource)

- **互斥轮流使用**：每个时刻只有一个进程使用，但不会耗尽，在宏观上各个进程轮流使用。
 - 如CPU、主存和辅存、I/O通道、外设、数据结构如文件、数据库和信号量。
- **有可能剥夺资源**：由高优进程剥夺低优进程，或OS核心剥夺进程。
- **问题**：如果被剥夺，代价是什么？



6.1 死锁概述

(5) 死锁发生的原因

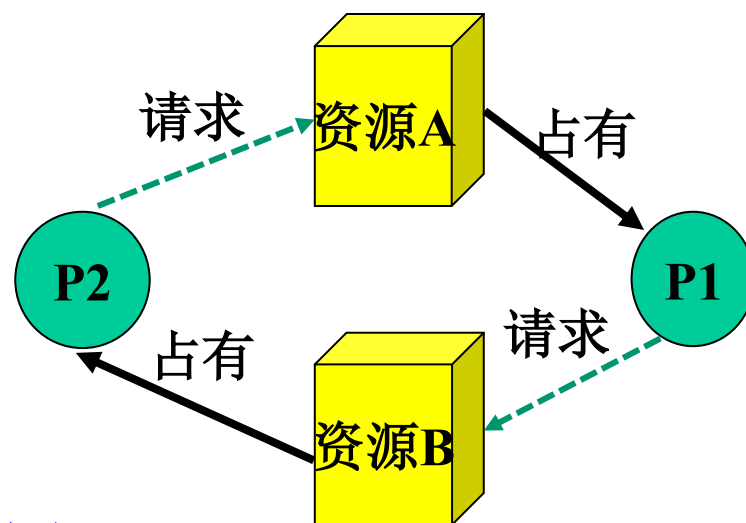
● 可重用资源死锁

P1

```
...  
Request(A);  
Request(B);  
Release(B);  
Release(A);
```

P2

```
...  
Request(B);  
Request(A);  
Release(A);  
Release(B);
```



死锁发生：双方都拥有部分资源，同时在请求对方已占有的资源。
如次序： P1<A> P2 P1 P2<A>

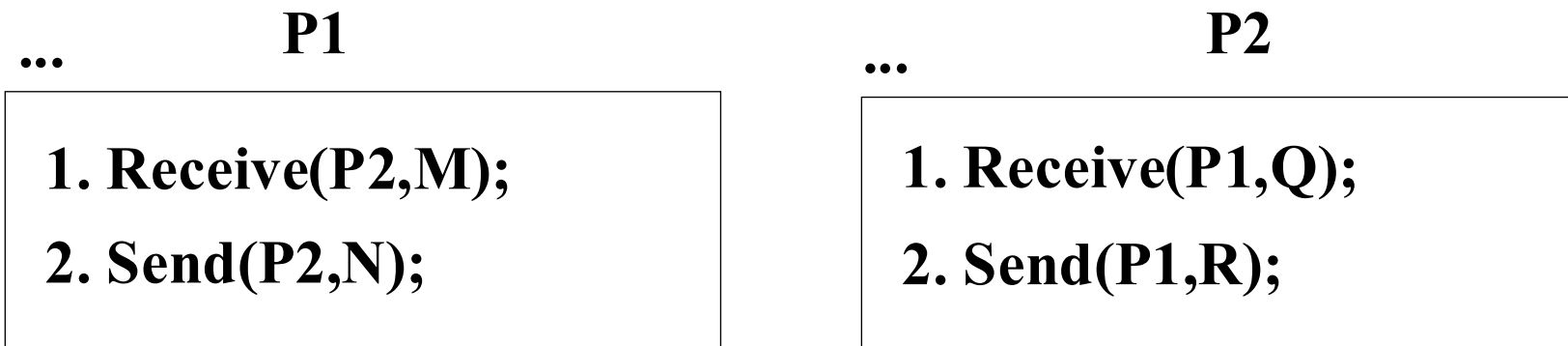


6.1 死锁概述

(5) 死锁发生的原因

● 可消费资源(consumable resource):

- 可以动态生成和消耗，一般不限制数量。但当进程得到一个资源时，该资源就不存在了。如硬件中断、信号、消息、缓冲区内的数据。



如次序: $P1 < 1 > P2 < 1 >$

死锁发生: 双方都等待对方去生成资源,

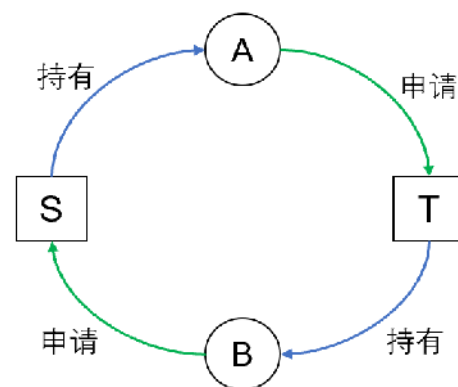


6.1 死锁概述

(6) 死锁发生的条件

只有**4个条件都满足时**，才会出现死锁。

- **互斥(Mutual exclusion)**: 任一时刻只允许一个进程使用资源;
- **请求和保持(Request and hold)**: 进程在请求其余资源时, 不主动释放已占用的资源;
- **非剥夺(Nonpreemptive)**: 进程已经占用的资源, 不会被强制剥夺;
- **环路等待(Circular Wait)**: 环路中的每一条边是进程在请求另一进程已经占有的资源。





6.1 死锁概述

(6) 死锁发生的条件

只有4个

● 互斥

资源

● 请求

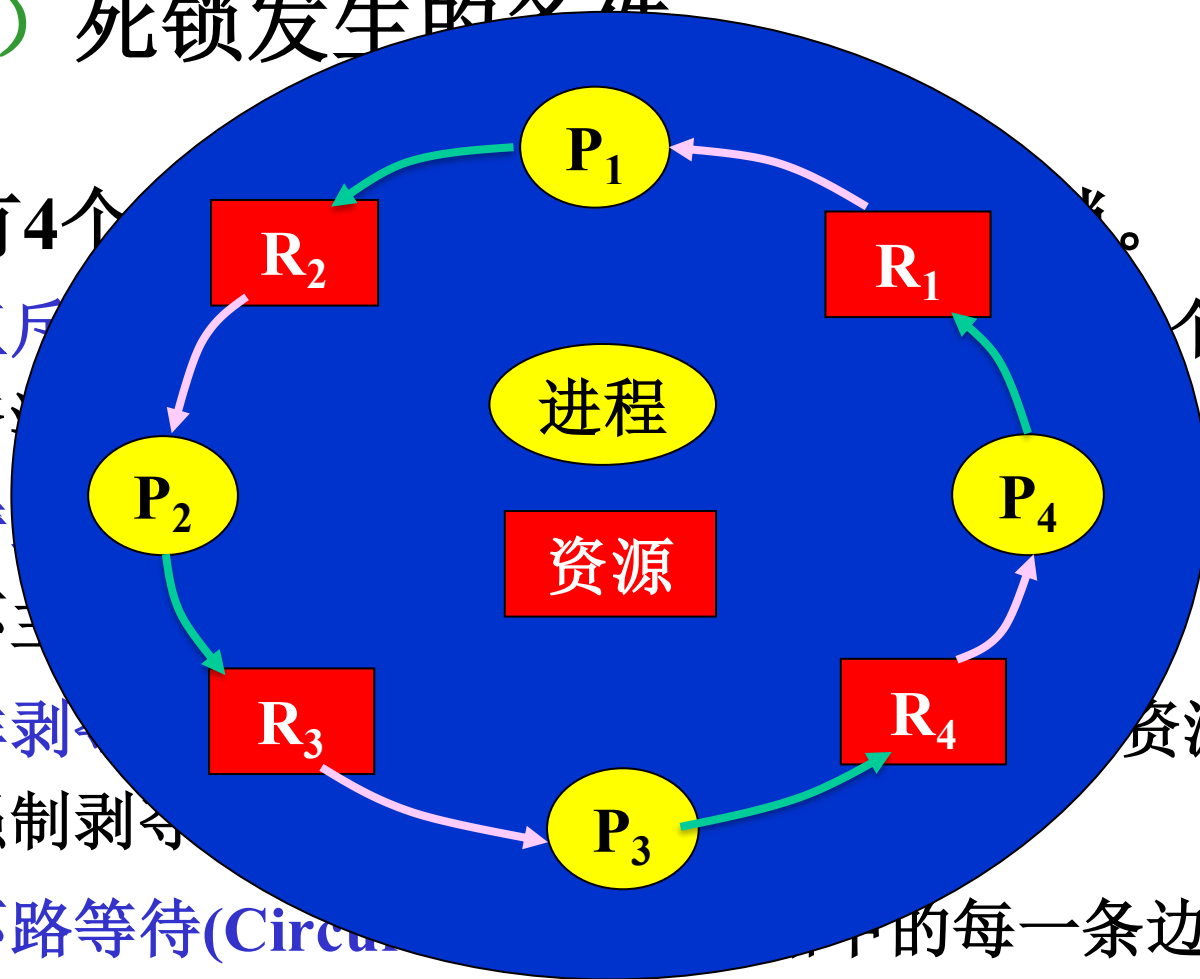
不三

● 非剥夺

强制剥夺

● 环路等待(Circular)

求另一进程已经占有的资源。



个进程使用

其余资源时，

资源，不会被

的每一条边是进程在请



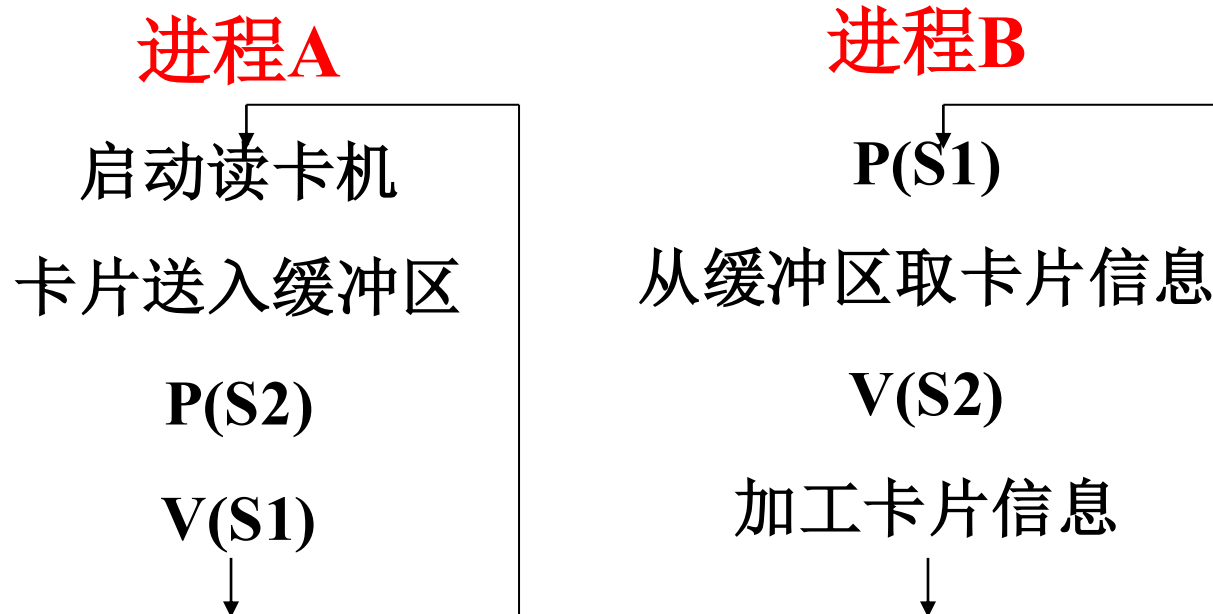
6.1 死锁概述

(6) 死锁发生的条件

死锁举例1

➤ P、V操作顺序不当引起的死锁

设信号量：S1、S2，初值为0



怎么会死锁？



6.1 死锁概述

(6) 死锁发生的条件

● 死锁举例2

➤ 资源共享引起的死锁

问题： m 个资源， n 个进程，若每个进程均要求 i 个资源，当 $m < ni$ 时如何保证不死锁？



● 保证不死锁的最少资源数为： $m = n(i-1) + 1$

● 为什么？

✓ 最坏的情况：每个进程均占有了 $i-1$ 个资源



6.1 死锁概述

(7) 处理死锁的基本办法【后面详讲】

方法	资源分配策略	各种可能模式	主要优点	主要缺点
预防 Prevention	保守的；宁可资源闲置（从机制上使死锁条件不成立）	一次请求所有资源<条件2:请求与保持>	适用于作突发式处理的进程；不必剥夺	效率低；进程初始化时间延长；剥夺次数过多；多次对资源重新启动；不便于灵活申请新资源
		资源剥夺<条件3:不剥夺条件>	适用于状态可以保存和恢复的资源	
		资源按序申请<条件4:环路条件>	可以在编译时（而不必在运行时）进行检查	
避免 Avoidance	是“预防”和“检测”的折中（在运行时判断是否可能死锁）	寻找可能的安全的运行序列	不必进行剥夺	使用条件：必须知道将来的资源需求；进程可能会长时间阻塞
检测 Detection	宽松的；只要允许，就分配资源	定期检查死锁是否已经发生	不 延长进程初始化时间；允许对死锁进行在线处理	通过剥夺解除死锁，造成损失



6.1 死锁概述

小结:

- 死锁的现象
- 定义
- 原因
- 必要条件
- 死锁的解决方法
 - 预防、避免、检测



6.2 死锁预防(Deadlock Prevention)

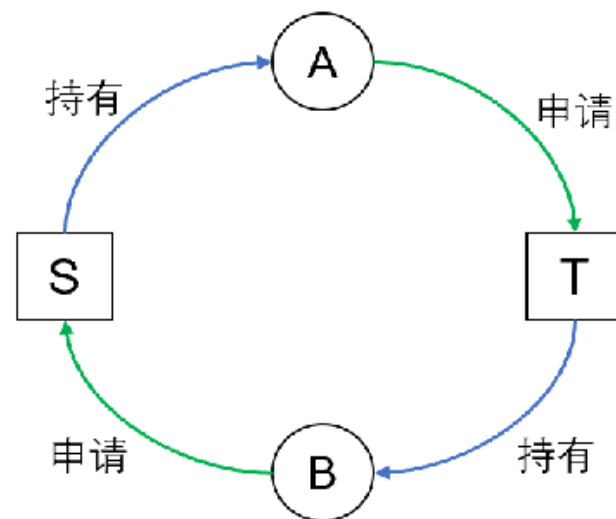


➤ **预防**是采用某种策略，**限制**并发进程对资源的请求，使系统在任何时刻都**不满足死锁的必要条件**。

- ① 互斥条件
- ② 请求与保持条件
- ③ 不剥夺条件
- ④ 环路条件

➤ 预防死锁的两种策略：

- 预先静态全分配法
- 有序资源使用法





6.2 死锁预防(Deadlock Prevention)



(1) 预先静态全分配法:

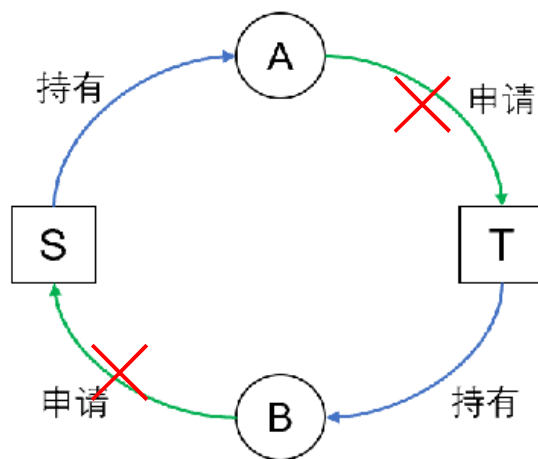
✓ 策略: 运行前预先分配所需全部资源, 保证不等待资源;

✓ 目标: 针对死锁的第2个条件 (请求与保持条件)

✓ 问题:

➤ 降低了对资源的利用率, 降低进程的并发度;

➤ 有可能无法





6.2 死锁预防

(2) 有序资源使用法

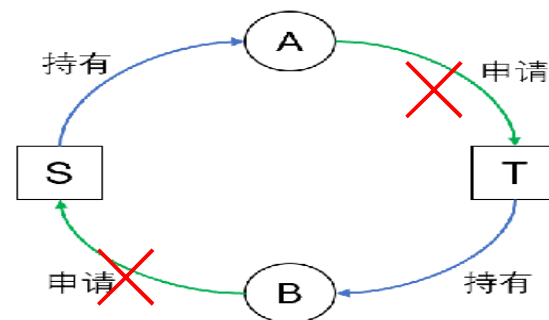
- 策略：把资源分类按顺序申请，例如：

$$R1 < R2 < \dots < Ri < Rj < \dots < Rn.$$

- 目标：保证不形成环路，针对死锁的第4个条件
- 现象：如果一个进程已经分配了 R_i 类型的资源，它接下来请求的资源只能是那些排在 R_i 类型之后的资源；
- 例如： P_a 获得 R_i ，要请求 R_j ；而 P_b 获得 R_j ，请求 R_i
 - P_a 是可以满足的，但 P_b 不能满足，因为资源 R_i 排在 R_j 前面。

- 问题：

- 限制进程对资源的请求，增加了程序员的负担；
- 资源的排序占用系统开销；



假设 $\dots < S < T < \dots$ ，则 B 不会申请 S



6.3 死锁的避免Avoidance



思路：在进程申请分配资源时，判断是否会出现死锁，如不会死锁，则分配资源。

● **定义：**在系统运行过程中，对进程发出的每一个**系统能够满足**的资源申请进行**动态检查**，并根据检查结果决定是否分配资源，若分配后系统可能发生死锁，则不予分配，否则予以分配。

● **有问题？明白吗？**

● **类比：**这个定义，跟现实生活中的什么很类似？



6.3 死锁的避免Avoidance



(1) 银行家算法 (Dijkstra, 1965)

- 一个银行家把他的**固定资金** (capital) 贷给若干顾客。
只要不出现一个顾客借走所有资金后**还不够**，银行家的资金应是**安全的**。银行家需一个算法保证**借出去的资金在有限时间内可收回**。
OS 资源 进程
- 假定顾客分成若干次进行贷款；并在第一次借款时，能说明他的**最大借款额**。
- 具体算法：
 - 顾客的借款操作依次顺序进行，直到全部操作完成；
 - 银行家对当前顾客的借款操作进行判断，以确定其**安全性**（能否支持顾客借款，直到全部归还）；
 - 安全时，贷款；否则，暂不贷款。



6.3 死锁的避免Avoidance

(1) 银行家算法 (Dijkstra, 1965)

● 银行家算法——例

假定系统有三个进程P1, P2, P3, 共有12台磁带机。进程P1总共要求10台磁带机, P2和P3分别要求4台和9台。设在T0时刻进程P1, P2, P3已分别获得5, 2, 2台, 尚有3台空余未分。

	最大需求	已分配	尚需	可用
P1	10	5	5	3
P2	4	2	2	
P3	9	2	7	



称 P2, P1, P3 为安全序列



6.3 死锁的避免Avoidance



(1) 银行家算法 (Dijkstra, 1965)

● 银行家算法——例

在前例基础上，P3提出申请2台资源，是否可以满足要求？

	最大需求	已分配	尚需	可用
P1	10	5	5	1
P2	4	2	2	
P3	9	4	5	

解：先假设能够满足要求且分配资源，则系统状态如上：

只剩余 1 台资源，不能使任何一个进程执行结束，因而不存在安全序列，所以系统不安全，因此拒绝分配资源，撤销开始的假设。



6.3 死锁的避免Avoidance



(1) 银行家算法 (Dijkstra, 1965)

● 算法

数据结构:

n: 系统中进程的总数

m: 资源类总数

A: ARRAY[1..m] of integer; (Available: 空闲的)

U: ARRAY[1..n,1..m] of integer; (Used: 正在使用的)

N: ARRAY[1..n,1..m] of integer; (Need: 尚需的)

R: ARRAY[1..n,1..m] of integer; (Request:本次请求的)



6.3 死锁的避免Avoidance



(1) 银行家算法 (Dijkstra, 1965)

当进程 p_i 提出资源申请 R 时，系统执行下列步骤：

(1) IF $R[i] > A$ Then Goto (7);

(2) IF $R[i] > N[i]$ Then 错误返回;

(3) 假设为 p_i 分配资源，且修改状态为：

$A := A - R[i]$; $U[i] := U[i] + R[i]$; $N[i] := N[i] - R[i]$;

(4) 检测系统目前状态是否安全？(调用安全检查算法isSafe?)

(5) 如果安全，则 p_i 进程得到资源，返回；否则：

(6) 恢复系统状态到(3)之前的状态：

$A := A + R[i]$; $U[i] := U[i] - R[i]$; $N[i] := N[i] + R[i]$;

(7) 请求失败， p_i 等待

A: 空闲的
U: 正在使用的
N: 尚需的
R: 本次请求的



6.3 死锁的避免Avoidance



(2) 安全性检查算法—IsSafe()

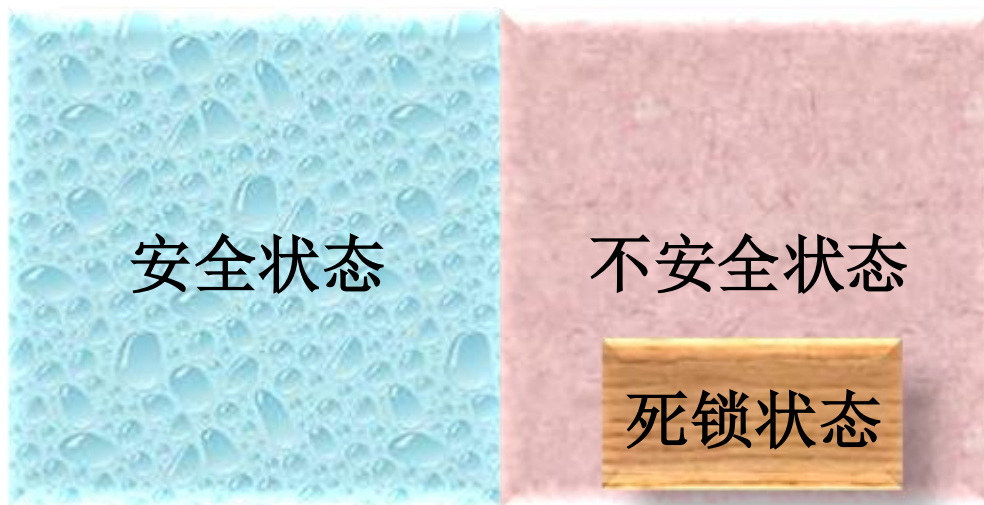
- **安全状态**: 如果存在一个由系统中所有进程构成的安全序列 $P_1, \dots, P_n$, 则系统处于安全状态。
- **安全序列**: 一个进程序列 $\{P_1, \dots, P_n\}$ 是安全的, 如果对于每一个进程 $P_j (1 \leq j \leq n)$, 它以后尚需要的资源量**不超过**系统当前剩余资源量**与**所有进程 $P_i (i < j)$ 当前占有资源量**之和**, 则系统处于安全状态。对于 P_1 而言, 当前剩余资源量应该能够满足它的尚需资源量。
- **安全状态一定是没有死锁发生的! 不安全状态一定会发生死锁吗?**
- **训练**: 给出上述描述的形式化公式 $N_j \leq A + \sum_{k=1, j-1} U_k$



6.3 死锁的避免Avoidance

(2) 安全性检查算法—IsSafe()

- **不安全状态**：不存在一个安全序列，不安全状态不一定导致死锁。





6.3 死锁的避免 Avoidance

(2) 安全性检查算法—IsSafe()

数据结构: **Work: ARRAY[1..m] of integer;**
Finish: ARRAY[1..n] of Boolean;

1. **Work = A;**

2. **For (i=1; i≤n; i++) Finish[i] = False;**

3. 查找满足下列条件的进程 i :

Finish[i] = False 且 $N[i] \leq \text{Work}$

4. i 找到了吗?

5. 如果没找到, 则转8

6. **Work = Work + U[i]; Finish[i] = True;**

7. **Goto 3;**

8. 所有j ($1 \leq j \leq n$) 的 **Finish[j]=True?**

9. 若是, 返回安全状态;

10. 否则, 返回不安全状态



6.3 死锁的避免Avoidance



(3) 银行家算法的特点

银行家算法的特点：

- 允许互斥、部分分配和不可抢占，可提高资源利用率；
- 要求**事先说明最大资源要求**，在现实中很困难；
- 考虑的进程必须是**无关的**，即它们的执行顺序必须没有任何**同步要求的限制**；也就是在找安全序列的过程中没有考虑进程之间的同步约束；
- 分配的资源数目必须是固定的。



6.3 死锁的避免Avoidance

(4) 银行家算法举例

设系统在T0时刻系统状态为：

	最大资源需求			已分配资源数量		
	A	B	C	A	B	C
P1	5	5	9	2	1	2
P2	5	3	6	4	0	2
P3	4	0	11	4	0	5
P4	4	0	5	2	0	4
P5	4	2	4	3	1	4

剩余资源向量为：

$A = (2, 3, 3)$

系统采用银行家算法实施死锁避免策略。

- ① T0时刻是否为安全状态？若是，请给出安全序列。
- ② 在T0时刻若进程P2请求资源 (0, 3, 4)，是否能实施资源分配？为什么？
- ③ 在②的基础上，若进程P4请求资源 (2, 0, 1)，是否能实施资源分配？为什么？



6.3 死锁的避免Avoidance

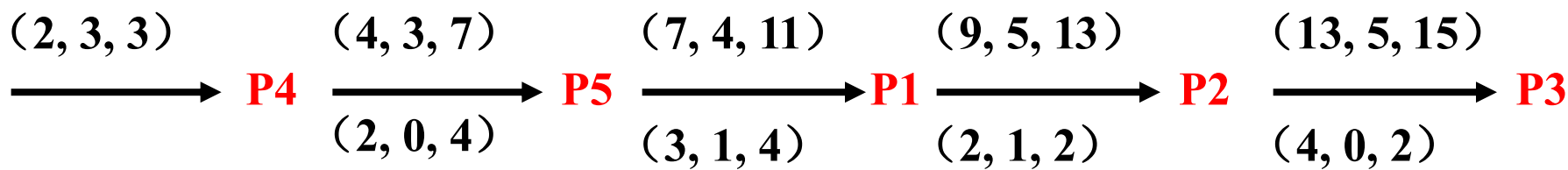
(4) 银行家算法举例

解:

	最大资源需求M			已分配资源数量U		
	A	B	C	A	B	C
P1	5	5	9	2	1	2
P2	5	3	6	4	0	2
P3	4	0	11	4	0	5
P4	4	0	5	2	0	4
P5	4	2	4	3	1	4

剩余向量A = (2, 3, 3)

① 是否安全?



安全序列: p4, p5, p1, p2, p3

思考: 安全序列是否唯一?



6.3 死锁的避免Avoidance



(4) 银行家算法举例

	最大资源需求M			已分配资源数量U			尚需资源N		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	2	0	4	2	0	1
P5	4	2	4	3	1	4	1	1	0

剩余向量 $\mathbf{A} = (2, 3, 3)$

②在T0时刻若进程P2请求资源 $(0, 3, 4)$ ，是否能实施资源分配？为什么？

解：假设能够分配，则P2还需 $(1, 0, 0)$ ，但因为
 $\mathbf{R}(0, 3, 4) > \mathbf{A}(2, 3, 3)$ ，所以不能满足。



6.3 死锁的避免Avoidance

(4) 银行家算法举例

	最大资源需求M			已分配资源数量U			尚需资源N		
	A	B	C	A	B	C	A	B	C
P1	5	5	9	2	1	2	3	4	7
P2	5	3	6	4	0	2	1	3	4
P3	4	0	11	4	0	5	0	0	6
P4	4	0	5	2	0	4	2	0	1
P5	4	2	4	3	1	4	1	1	0

剩余向量A = (2, 3, 3)

③ 在②的基础上，若进程P4请求资源(2, 0, 1)，是否能实施资源分配？为什么？

解：因为 $R(2, 0, 1) \leq N_4(2, 0, 1)$ 且 $< A(2, 3, 3)$ ，假设可以满足，则 $N_4=(0, 0, 0)$ ， $A=(0, 3, 2)$ ，在此基础上，

$(0, 3, 2) \xrightarrow{P4} (4, 3, 7) \xrightarrow{P5} (7, 4, 11) \xrightarrow{P1} (9, 5, 13) \xrightarrow{P2} (13, 5, 15) \xrightarrow{P3}$
所以是安全的，因此可以实施分配



6.4 死锁的检测与恢复

6.4.1 死锁检测

- **检测：** 允许死锁发生，OS不断监视系统进展情况，判断死锁是否发生。
 - 保存资源的请求和分配信息，利用某种算法对这些信息加以检查，以判断是否存在死锁。
 - 死锁检测算法主要是**检查是否有循环等待**。
- **解除：** 一旦死锁发生则采取专门的措施，解除死锁并以最小的代价恢复系统运行。
- **缺点：** 在解除死锁时，常常造成进程终止或重新启动（重新申请资源）



6.4 死锁的检测与恢复

6.4.1 死锁检测

检测时机

- ① 当进程等待时检测死锁，其缺点是系统的开销大。
- ② 定时检测。
- ③ 系统资源利用率下降时检测死锁。



6.4 死锁的检测与恢复

6.4.1 死锁检测——资源分配图

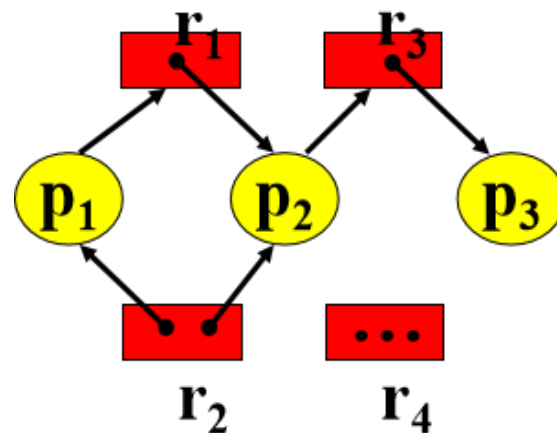
● $G=(V, E)$

● V : 顶点集合 E : 边集合

● 顶点集合分为: $P=\{p_1, p_2, \dots, p_n\}$ 表示系统中所有活动进程; $R=\{r_1, r_2, \dots, r_m\}$ 表示系统中全部资源类型

● 有向边 $p_i \rightarrow r_j$ 称为**申请边**, 表示进程 p_i 申请一个单位的 r_j 资源, 但当前 p_i 在等待该资源。

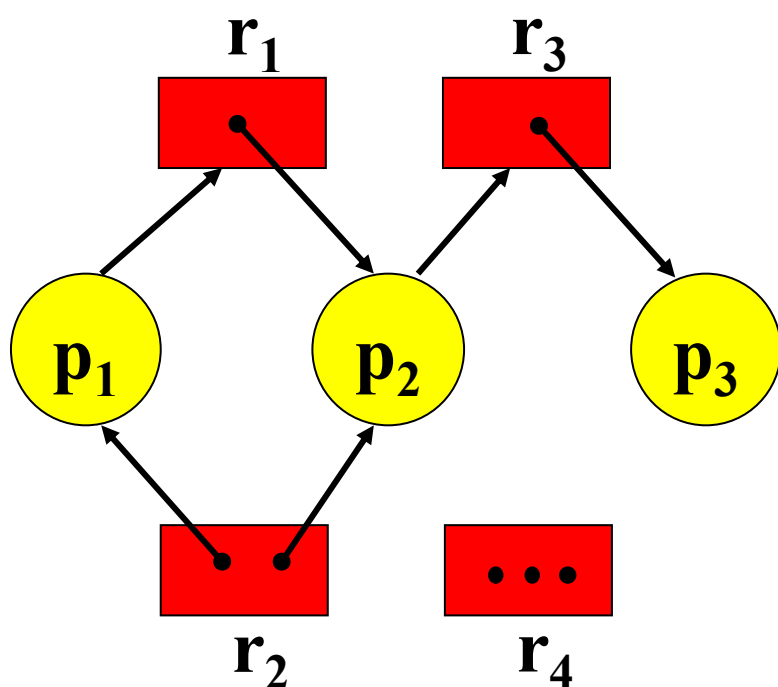
● 有向边 $r_j \rightarrow p_i$ 称为**赋予边**, 表示一个单位的资源 r_j 已经分配给进程 p_i 。





6.4 死锁的检测与恢复

6.4.1 死锁检测——资源分配图



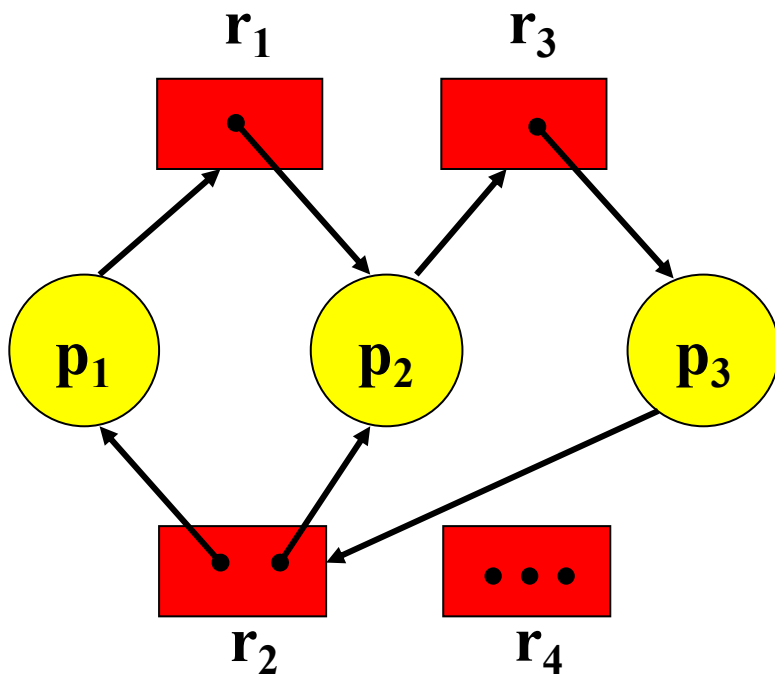
- p_1 占有一个 r_2 资源，且等待一个 r_1 资源
- p_2 占有一个 r_1 类资源和一个 r_2 类资源，等待一个 r_3 类资源
- p_3 占有一个 r_3 类资源

该图不含环路，系统中没有进程处于死锁状态



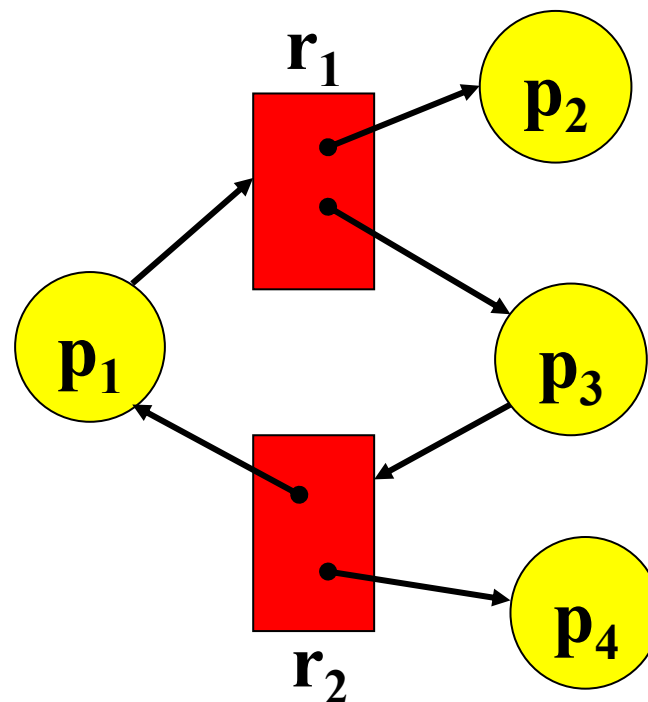
6.4 死锁的检测与恢复

6.4.1 死锁检测——资源分配图



有环路，有死锁吗？

有死锁的资源分配图



有环路，有死锁？

但无死锁的资源分配图

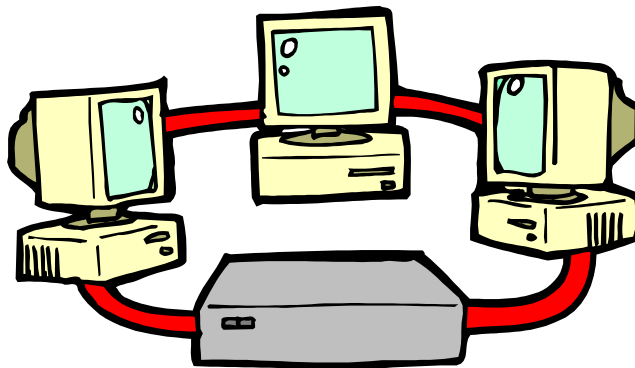


6.4 死锁的检测与恢复



6.4.1 死锁检测——资源分配图

- 如果资源分配图中**没有环路**，系统**不会陷入死锁状态**；
- 如果**存在环路**，那么系统可能出现死锁，**但不确定**。
- **关键是：**出现环路后，是否还有进程能够继续前进。

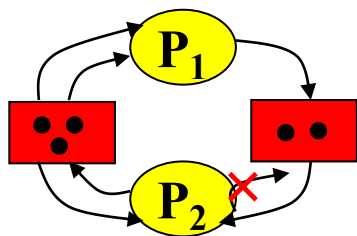




6.4 死锁的检测与恢复

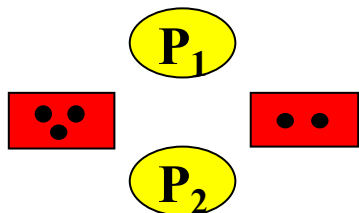
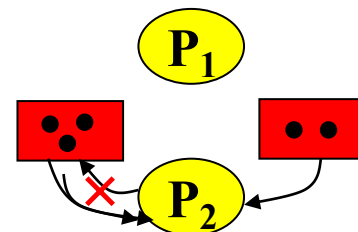


6.4.2 死锁检测——死锁定理



- 在图中找出一个既不阻塞又非独立的进程结点 P_i 。在顺利情况下, P_i 可获得所需资源而继续运行, 直至运行完毕, 再释放所占有的全部资源, 这相当于消去 P_i 所有的请求边和分配边, 使之成为孤立的结点。

- P_1 释放资源后, 便可使 P_2 获得资源而继续运行, 直至 P_2 完成后又释放出它所占有的全部资源。



- 在进行一系列简化后, 若能消去图中所有的边, 使所有的进程结点都成为孤立结点, 则称该图是可完全简化的; 若不能通过任何过程使该图完全简化, 则称该图是不可完全简化的。



6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

- 对于较复杂的资源分配图，可能同时有多个既未阻塞又非孤立的进程结点。存在多个简化顺序
- 有关文献已经证明，所有的简化顺序，都将得到相同的不可简化图；
- 死锁定理：
 - S状态为死锁状态的条件是：当且仅当S状态的资源分配图是不可完全简化的。



6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

● 数据结构

- **Available:** 初始可用资源向量
- **Work:** 当前可用资源向量
- **Request:** 进程申请资源向量
- **Allocation:** 分配资源向量
- **L:** 孤立结点集合

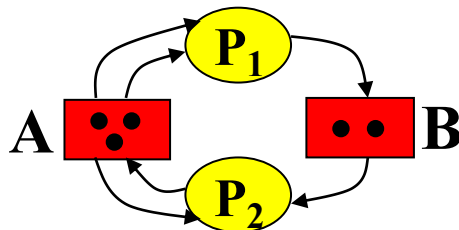
$$\text{Available}=[3 \ 2]$$

$$\text{Work}=[0 \ 1]$$

$$\text{Request}=\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$\text{Allocation}=\begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$$

$$L=\{\}$$





6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

● 步骤

- ① 将不占用资源的进程，记入L表中；
- ② 从进程集合中找到一个 $\text{Request}_i \leq \text{Work}$ 的进程：
- ③ 回收 p_i 的资源，简化资源分配图，维护工作向量Work：
 $\text{Work} := \text{Work} + \text{Allocation}[i]$
- ④ 将 p_i 记入L
- ⑤ 重复该过程，若不能将所有进程都记入L中，便表明系统状态S的资源分配图是不可完全简化的，该状态将发生死锁。

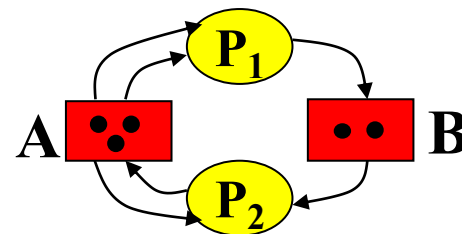


6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

● 步骤

- ① 将不占用资源的进程记入L表中;
- ② 从进程集合中找到一个 i :
 $\text{Request}_i \leq \text{Work}$ 的进程:
- ③ 将其资源分配图简化, 释放资源, 维护工作向量 $\text{Work} := \text{Work} + \text{Allocation}_i$
- ④ 将它记入L
- ⑤ 重复该过程, 若不能将所有进程都记入L中, 便表明系统状态S的资源分配图是不可完全简化的, 该状态将发生死锁。



$\text{Available} = [3 \ 2]$

$\text{Work} = [0 \ 1]$

$\text{Request} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

$\text{Allocation} = \begin{bmatrix} 2 & 0 \\ 1 & 1 \end{bmatrix}$

$L = \{\}$

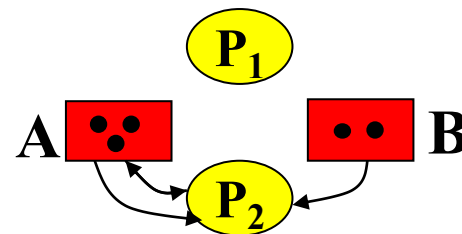


6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

步骤

- ① 将不占用资源的进程记入L表中;
- ② 从进程集合中找到一个 $Request_i \leq Work$ 的进程:
- ③ 将其资源分配图简化, 释放资源, 维护工作向量 $Work := Work + Allocation_i$
- ④ 将它记入L
- ⑤ 重复该过程, 若不能将所有进程都记入L中, 便表明系统状态S的资源分配图是不可完全简化的, 该状态将发生死锁。



$Available = [3 \ 2]$

$Work = [0 \ 1]$

$Request = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$

$Allocation = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$

$L = \{P_1\}$

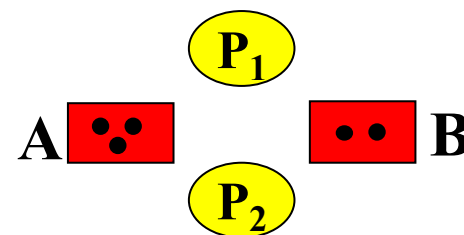


6.4 死锁的检测与恢复

6.4.2 死锁检测——死锁定理

● 步骤

- ① 将不占用资源的进程记入L表中;
- ② 从进程集合中找到一个 $Request_i \leq Work$ 的进程:
- ③ 将其资源分配图简化, 释放资源, 维护工作向量 $Work := Work + Allocation_i$
- ④ 将它记入L
- ⑤ 重复该过程, 若不能将所有进程都记入L中, 便表明系统状态S的资源分配图是不可完全简化的, 该状态将发生死锁。



$Available = [3 \ 2]$

$Work = [3 \ 2]$

$Request = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$

$Allocation = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$

$L = \{P_1\} P_2\}$



6.4 死锁的检测与恢复



6.4.3 死锁的恢复Deadlock Recovery

重要的是以最小的代价恢复系统的运行

- 通过撤消代价最小的进程或使撤消进程数目最小，以解除死锁。
- 挂起某些死锁进程，并抢占它的资源，以解除死锁。如：
 - 1) 重新启动
 - 2) 撤消进程
 - 3) 剥夺资源
 - 4) 进程回退



6.4 死锁的检测与恢复

6.4.3 死锁的恢复Deadlock Recovery

● 撤消进程的原则

- 进程优先级;
- 系统重启进程的运行代价



6.5 解决死锁问题的综合方法



- **资源归类**：将各种资源归入若干个不同的资源类(resource group)中，如：外存交换区空间，进程资源（可分配的设备，如磁带机，文件），主存空间，内部资源（如I/O通道）；
- **资源排序**：为防止在不同的资源类间由于循环等待产生死锁，可在不同资源类之间**规定次序**，对不同资源类中的资源采用线性按序申请的方法；
- **针对性优化**：对同一资源类中的资源，采用适当的方法，例如：
 - **外设资源**：使用**避免**的方法处理
 - **存储资源**：则采用**预防**方法。



6.5 解决死锁问题的综合方法



● 同类资源组的优化策略

- **外存交换区空间**：通过要求**一次性分配**所有的请求的资源来预防死锁，如果知道进程所需的最大存储需求（通常情况下是可知的），死锁避免是可以的
- **进程资源**：采用**死锁避免策略**很有效，因为进程可以事先声明它们将需要的这类资源，采用**资源排序**的策略也是可以的。
- **主存**：采取基于**剥夺的预防策略**。当一个进程被剥夺后，它仅仅被换到辅存，释放空间即可解决死锁
- **内部资源**：使用基于**资源排序**的预防策略。



6 死锁问题-总结(1/2)

- 小结
 - 死锁定义：一种状态
 - 原因：资源有限
 - 必要条件：互斥、请求与保持、非剥夺、循环等待
 - 死锁的解决办法（见下页）：
- 联想：死锁 vs 交通堵塞 --> 遵守交通法则



6 死锁问题-总结(2/2)



方法	资源分配策略	各种可能模式	主要优点	主要缺点
预防 Prevention	保守的；宁可资源闲置（从机制上使死锁条件不成立）	一次请求所有资源<条件2:请求与保持>	适用于作突发式处理的进程；不必剥夺	效率低；进程初始化时间延长；剥夺次数过多；多次对资源重新启动；不便于灵活申请新资源
		资源剥夺<条件3:不剥夺条件>	适用于状态可以保存和恢复的资源	
		资源按序申请<条件4:环路条件>	可以在编译时（而不必在运行时）进行检查	
避免 Avoidance	是“预防”和“检测”的折中（在运行时判断是否可能死锁）	寻找可能的安全的运行序列	不必进行剥夺	使用条件：必须知道将来的资源需求；进程可能会长时间阻塞
检测 Detection	宽松的；只要允许，就分配资源	定期检查死锁是否已经发生	不 延长进程初始化时间；允许对死锁进行在线处理	通过剥夺解除死锁，造成损失

1. 系统发生死锁时, 其资源分配图中必然存在环路. 因此, 如果资源分配图中存在环路, 则系统一定出现死锁.

- ☐ A 一定
- ☐ B 不一定
- ☐ C 不会

提交

2. P, V操作不仅可以实现并发进程之间的同步和互斥, 而且能够防止系统进入死锁状态. ()

- ☐ A 对
- ☐ B 错
- ☐ C 不确定

提交



6 死锁问题 (DEADLOCK)



作业

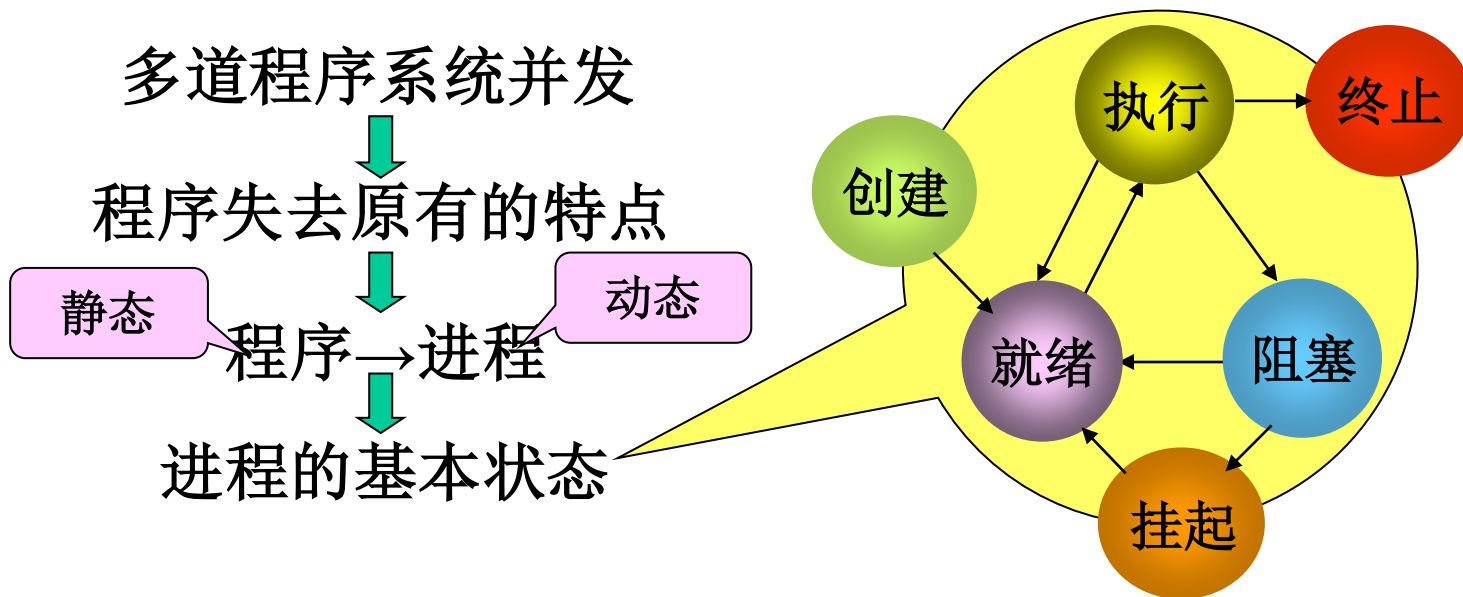
	已分配的资源			最大需求量			剩余资源:		
	A	B	C	A	B	C	A	B	C
P ₁	0	1	0	7	5	3			
P ₂	2	0	0	3	2	2	3	3	2
P ₃	3	0	2	9	0	2			
P ₄	2	1	1	2	2	2			
P ₅	0	0	2	4	3	3			

画出资源分配图，此状态是否为安全状态，如果是，则找出安全序列；在此基础上，请回答：

- ① P₂ 申请 (1, 0, 2) 能否分配？为什么？
- ② P₅ 申请 (3, 3, 0) 能否分配？为什么？
- ③ P₁ 申请 (0, 2, 0) 能否分配？为什么？

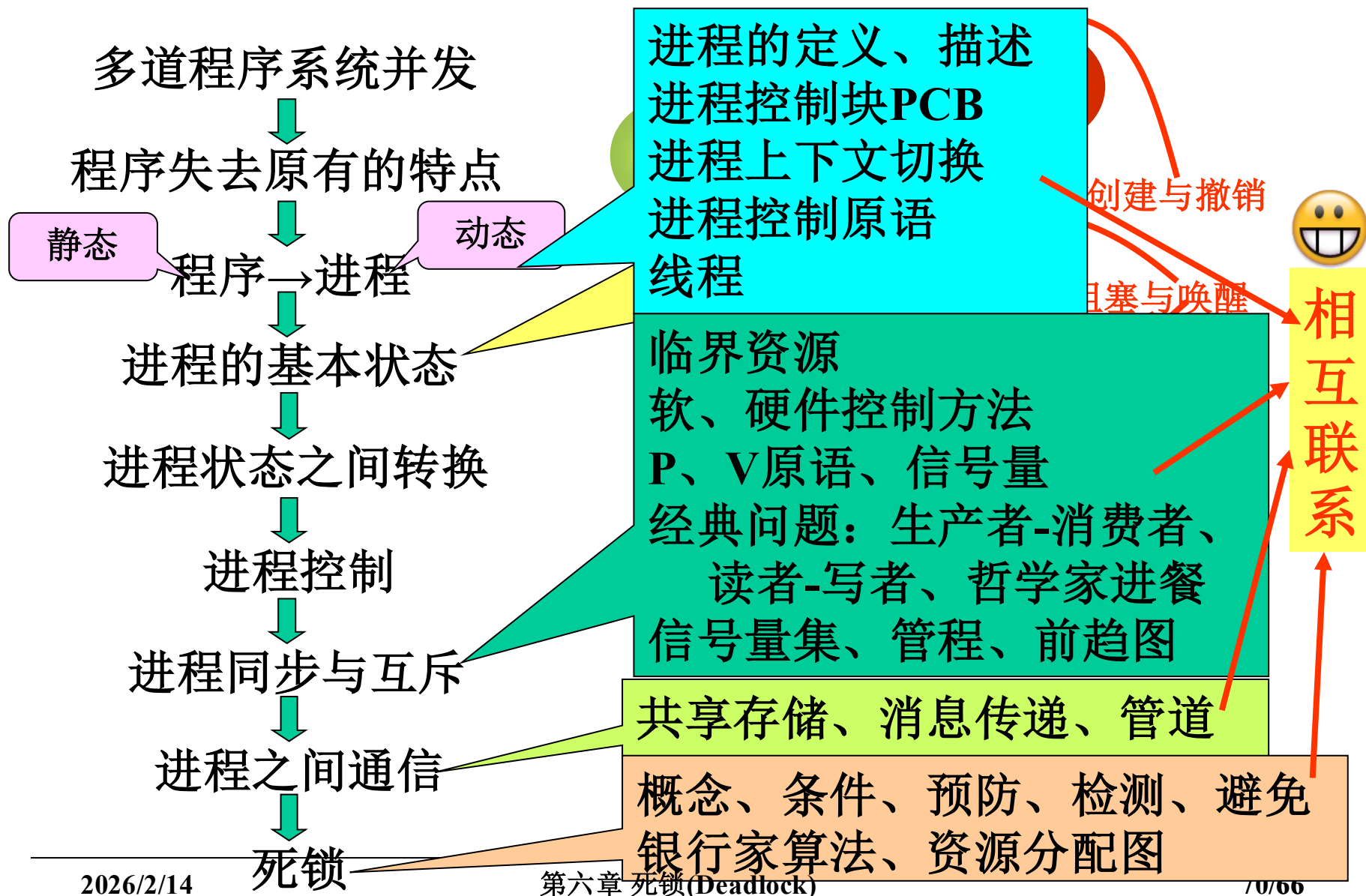


第六章 小结





第六章 小结





继续学习第7章： 处理机调度

